

B09 - AI-Assisted Multithreaded Mandelbrot Viewer with Zoom, Export, and Path Animation Goal

Use AI to help design and implement a multithreaded Mandelbrot set viewer that supports interactive zooming, image export, and animation along a user-defined path into the fractal (e.g. zoom flight), leveraging concurrency to render efficiently.

Tasks

1. Design & Implement: Use AI to assist in implementing the Mandelbrot renderer (coloring, iteration control) with multithreaded rendering, an interactive UI for panning/zooming, image export (e.g. PNG), and an animation feature that interpolates a path of viewpoints and generates frames or a video-like sequence.
2. Test & Demonstrate: Demonstrate interactive usage (smooth zooming and navigation), validate correctness of the fractal rendering, measure/observe the impact of multithreading, and showcase at least one non-trivial animation path into the fractal.
3. AI Usage & Verification Report: Collect and submit the prompts you used with AI and write a short report explaining how you evaluated and corrected the AI-generated code (thread-safety, performance, correctness of the math, and quality of exported images/animations).

Design phase

I already had an idea on how I wanted to implement this project, here's a summary of the technical requirements (called rules for the LLMs).

Rule #1

The first requirement is not to use third party libraries. One of the benefits of using AI to speedup development is that, for simple and non critical use cases (I would never reimplement something like OpenSSL) it is almost free to reinvent the wheel and avoid the dependency hell that plagues modern software development¹.

Rule #2

Implement the project using a client-server architecture to decouple the frontend and the backend

Rule #3

For the frontend I decided to implement a minimal web app so that with one codebase I could target any device with a modern browser. To limit the boilerplate at the minimum I opted for plain HTML + JS + CSS instead of framework like React, which is suitable for more complex application with a complex state management.

Rule #4

For the backend I wanted to be able to easily use SIMD instructions to speedup the rendering and to implement GPU rendering, so the candidates were C, C++, Rust and Zig. In Table 1 the evaluation points can be found. Ultimately, I opted for C because I worked with it professionally and because I prefer it for simple projects that require high performance and low level programming².

¹https://en.wikipedia.org/wiki/Dependency_hell.

²And I don't like Rust :)

Language	Pros	Cons
C	• Simple • Well-known by LLMs • Stable SIMD support via <code>immintrin.h</code> • OpenGL support	• Manual memory management • Rudimental thread support
C++	• Well-known by LLMs • Support for promises and futures • Stable SIMD support via <code>immintrin.h</code> • OpenGL support	• Good for big enterprise projects, but unnecessary complex for smaller projects
Rust	• Memory safe • Safe concurrency support via Arc wrappers and async multithreaded tokio runtime • Compiler hints help the LLMs (and agents) to iterate and fix the code faster	• SIMD support is still experimental • Harder and more complex to write • OpenGL needs unsafe blocks
Zig	• Built-in advanced memory management via allocators • Better memory safety via <code>Optional</code> type and <code>defer/errdefer</code> mechanism • SIMD support out-of-the-box via <code>@Vector</code> • OpenGL support via C interop functionality	• It is still a relatively new language, so I do not expect LLMs to be good at handling more complex tasks • I have zero experience with this language

Table 1: Comparison between languages

Implementation

Chat based

To structure the project and set the foundation of the renderer and the visualizer, I used **Gemini 3 Pro** since it is free to use with the @unipi account.

Hello I have a implement this college project project:

Multithreaded Mandelbrot Viewer with Zoom, Export, and Path Animation along a user-defined path into the fractal (e.g. zoom flight), leveraging concurrency to render efficiently.

I have a basic idea on how to architecture this:

- *Frontend: Basic HTML + JS + CSS to be able to view the mandelbrot. In a first iteration hw acceleration is not mandatory. We will discuss this later if there is time.*
- *Backend: I want to leverage SIMD (Intel AVX/AVX2) instructions to speedup the computation, other than using multithreading, so I was thinking about a C server that does the heavy-lifting of the computation.*

Now, the two components have to talk each other so I was thinking about websockets or better webRTC, but I can evaluate other options if you think are worth it.

An important rule: this has to be a ZERO dependency project, everything must be implemented from scratch, even the websocket/webrtc communication between the backend and the frontend

Please evaluate this proposal, highlight the strong and weak points and help me write down the implementation roadmap.

Gemini basically approved the tech stack and the constraints, suggesting to use websockets as it's easier to implement than WebRTC.

To visualize the madelbrot it suggested the canvas API (<canvas> tag) and it also designed the basic communication protocol:

- The client sends [x_min,y_min,x_scale,y_scale,width,height,iterations] as a CSV string (I added the iteration count later as it was fixed).
- The server responds with an ArrayBuffer containing the bitmap to be painted on the <canvas>.

Let's start with the math part. Please create a render_simd function to leverage AVX instruction, conditionally for the correct target architecture (I have to test it on my laptop with AVX2 and on my server with AVX) and create a main function to benchmark the speedup when using single core, multicore and multicore + SIMD

Created successfully a benchmark program to test the every combination (No SIMD single/multithreaded, SIMD single/multithreaded).

Go on implementing the WebSocket accept function, implementing the base64 encode and SHA1 checksum from scratch.

Created the two utility functions correctly and the library to upgrade an HTTP connection to a websocket, implementing also the send and receive functions.

Please finish the implementation creating the main server loop to accept requests from multiple clients and add a threadpool to render the mandelbrot.

It completed the backend application with the main loop so that I could start implementing the frontend.

At this point I started a new chat to prevent the LLM from hallucinating as it can happen with longer chats.

A bit of context about the project

Let's start the frontend implementation. These are the rules:

- Zero dependency: the frontend shouldn't require any external library
- Use pure HTML5 + JS + CSS, no frameworks.
- The style should be minimal and modern (dark theme Shadcn like)
- Remember the specification: Mandelbrot renderer with coloring, iteration control, an interactive UI for panning/zooming, image export (e.g. PNG), and an animation feature that interpolates a path of viewpoints and generates frames or a video-like sequence
- The protocol to be used over the websocket is:

```
// Expected format:
"x_min,y_min,x_scale,y_scale,width,height,iterations"
sscanf(payload, "%lf,%lf,%lf,%lf,%d,%d,%d", &x_min, &y_min, &x_scale,
        &y_scale, &width, &height, &iterations)
```

- The data is returned as an ArrayBuffer so the receive part should look something like this:

```
const socket = new WebSocket(`ws://${baseUrl}:8080`);
socket.binaryType = 'arraybuffer';

socket.onopen = () => {
  // Send initial coords
  socket.send("-2.0,-1.5,0.005,0.005,800,600");
};

socket.onmessage = (event) => {
  const arrayBuffer = event.data;
  const uint8Array = new Uint16Array(arrayBuffer);
  // Put this data into an HTML5 Canvas ImageData
};
```

It created the index.html, the style.css and the main application script.js. The application opened in a browser seems to be fine, apart from minor UI inconsistencies that I manually fixed very quickly.

Now that I have to test the whole architecture, I started a new chat asking for a simple Makefile to compile the project.

Can you create a makefile based on this structure to compile the backend?

```
.
├── Makefile
└── src
    ├── include
    │   ├── mandelbrot.h
    │   ├── tpool.h
    │   ├── websocket.h
    │   └── websocket_utils.h
    ├── mandel_bench
    ├── mandelbrot.c
    ├── server.c
    ├── tpool.c
    ├── websocket.c
    └── websocket_utils.c
```

The Makefile was almost ok, except for a couple of compilation flags such as `-mfma` and `-mavx2`, which prevented the code from compiling on the older architecture of my server, so I changed them to `-march=native`, enabling all the optimization available on the machine that compiles the code, since the `render_simd` function automatically compiles to use the best available SIMD instruction set.

First troubleshooting

The code, unexpectedly compiled right away and the frontend connected to the backend successfully, but the mandelbrot was not there and the backend crashed with a `segfault` a moment after the connection.

Inspecting the code manually revealed two problem:

1. The buffer is declared as an array of 4096 elements and set to 0, but on line 10, the read function is allowed to overwrite the null terminator in the last position of the array. Since the buffer is later manipulated by `string.h` functions that expect the input string to be null terminated, I changed the buffer declaration to `char *buffer[4097]`. However, this was not the bug that made the backend crash.

```
1  char *buffer[4096] = {0};
2  // ...
3  client_fd =
4      accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
5  if (client_fd < 0)
6      continue;
7
8  printf("Client connected\n");
9  memset(buffer, 0, 4096);
10 read(client_fd, buffer, 4096);
11 printf("Received: %s\n", buffer);
12 if (perform_handshake(client_fd, buffer) != 0) {
13     close(client_fd);
14     continue;
15 }
16 printf("Handshake successful\n");
```

2. I manually isolated the function that caused the crash and it was the SHA1 implementation, where an overflow when computing `part_len` caused the `memcpy` try to copy terabytes of data. The LLM provided an update version that resolved the issue.

```
1 void sha1(const unsigned char *data, size_t len, unsigned char *hash) {
2     size_t i, j;
3     for (i = 0; i < len + 9; i += 64) {
4         unsigned char block[64] = {0};
5         size_t part_len = (i + 64 > len) ? len - i : 64;
6         memcpy(block, data + i, part_len);
7         // ...
8     }
9     // ...
```

At this point, the core of the project is working and I can focus on details and new features.

The agentic develop

Once the project worked as it should I took some time to review and audit the code:

- Check for redundancies and unused code
- Reduced the verbosity of some functions

Then, I containerized the application with docker to easily build and test it with few commands and also provide a ready to use configuration via environment variables. The project is now made of two parts:

- The container running the backend
- The container running Nginx to serve the frontend and proxy the requests to the backend

```
1 services:
2   backend:
3     build: ./backend
4     restart: unless-stopped
5     environment:
6       - MANDELBROT_THREADS=12 # Same as logical cores
7       - MANDELBROT_SIMD=1 # Use AVX instructions set
8     expose:
9       - 8080
10
11   frontend:
12     image: nginx:latest
13     restart: unless-stopped
14     ports:
15       - 8080:80
16     volumes:
17       - ./nginx:/etc/nginx
```

Code 1: First docker-compose.yml

At this point, I had a reproducible environment to manually test the application and the instructions to let the agent build and run the application, so the next step was to initialize a git repository to track the changes made by the agent and create a new branch, which I named `feature/opengl-support` as I wanted to implement the support for rendering the mandelbrot using the GPU. I opted for OpenGL because is well supported by the C language via `CHECK LIB NAME` and it can target both the Intel and NVIDIA GPUs with just one implementation.

First, I attempted using Opencode wired to a local instance of qwen3:8b running on Ollama on one of my servers, but the GTX1070 with only 8Gb of VRAM was not enough to get a good development experience, resulting in an high waiting time before the first token and an output that was not on par with much larger models. Then I tried to connect my Gemini account on both Opencode and Gemini CLI, which worked flawlessly.

Before issuing any task to the agent, I took some time to wrap things up with the two main Gemini chats I used to create the foundations of the project to create a `docs.md` file with all the implementation decisions took and a `specs.md` file with all the constraints and requirements imposed by both the specification and me.

I created a new specification file that outlined the new feature to be developed, specifying the function signature, the new environment variable to be used to enable GPU rendering and the regression and acceptance tests³ that the agent should perform in order to decide whether the implementation was acceptable or not. The agent successfully completed these steps:

```
> Please read @docs.md and @specs.md to get a context of what we're doing and then read @task.md to implement the OpenGL support
```

```
# Todos
[✓] Add OpenGL headers to mandelbrot.h
[✓] Implement render_opengl function in mandelbrot.c
[✓] Add GPU environment variable support
[✓] Update Makefile to link OpenGL libraries
[✓] Configure docker-compose.yml for GPU passthrough
```

At the first iteration it created the harness to make the function work, but the actual implementation falls back to the scalar rendering, so i decided to hand the task over to gemini cli which is offered with the free premium account

After a bit of back and forth directing the agent towards the solution, it finally make it work with opengl. Testing it with an intel and nvidia graphics, it works.

Now i tried to give him a spec file

```
## 2. Next features
```

As of now the project is composed by a frontend and a backend. The frontend is served by an nginx instance and consists in 3 static files:

```
- index.html
- script.js
- style.css
```

Now we want to:

```
- [x] implement a webserver in our backend.
- [x] make the backend serve both the mandelbrot generation and the webpages, being aware to implement the basic security features such as avoid use after free, no buffer overflows and so on.
- [x] the amount of requests is small so the webserver should run in its own thread with its own request queue.
- [x] The page should be served from `/var/www` and the requests should not escape from that path.
```

³Basically running the backend benchmark and then start the server and send some rendering requests using a python script.

A note on the backend project structure:

```
.
├─ benchmark.log <- ignore it
├─ Dockerfile <- dockerfile without nvidia framework
├─ Dockerfile.nvidia <- dockerfile with nvidia framework
├─ frontend <- frontend directory
│   ├── index.html
│   ├── script.js
│   └── style.css
├─ Makefile <- makefile for compilation
├─ plot_benchmark.py <- ignore it
├─ src <- C backend source
│   ├── include <- headers
│   ├── mandelbrot.c <- mandelbrot rendering
│   ├── server.c <- entrypoint and server implementation
│   └── tpool.c <- mandelbrot threadpool implementation (for the webserver use another thread)
│   ├── webserver.c <- Implement webserver HERE
│   ├── websocket.c <- websocket handshake and send/recieve
│   └── websocket_utils.c <- websocket utils functions
└─ test_one_frame.py
```

And it implemented all the features. Those have been reviewed using `git diff` and it successfully implemented a basic webserver to serve static files:

- First it handles websockets, checking for a successful handshake
 - No handshake -> check if it is a get request and in that case serve it
 - It is an handshake -> render the request and send it back

Appendix: Lesson learned and concepts

Websockets from scratch

WebSockets provide a full-duplex communication channel over a single TCP connection. The protocol begins with an HTTP handshake:

1. The client sends an upgrade request containing a `Sec-WebSocket-Key`.
2. The server takes this key, appends a specific GUID string, computes the SHA-1 hash, and returns the base64-encoded result in the `Sec-WebSocket-Accept` header.
3. Once established, data is exchanged in frames. Each frame contains control bits (fin, opcode), a payload length (which can be extended), a masking key (for client-to-server messages), and the payload itself.

In this project, the implementation is handled in `src/websocket.c` and does not rely on external libraries:

- **Handshake:** The `perform_handshake` function searches for the `Sec-WebSocket-Key` header, concatenates it with the magic GUID `258EAF5E-E914-47DA-95CA-C5AB0DC85B11`, computes the SHA-1 hash, and Base64 encodes the result to complete the upgrade.
- **Frame Parsing:** `receive_frame` interprets the first two bytes to determine the opcode (checking for connection close `0x8`) and payload length. It handles variable-length payloads (7-bit, 16-bit extended, or 64-bit extended) and unmask the client data using the 4-byte masking key (`buffer[i] ^ mask_key[i % 4]`).

- **Binary Responses:** The `send_binary_frame` function constructs valid WebSocket frames with the FIN bit set and opcode `0x2` (Binary), ensuring large image buffers are transmitted correctly by setting the appropriate extended payload length bytes in big-endian order.

Webserver from scratch

Since I've never implemented a Webserver from scratch in C (only in Java), I decided to implement it since the features required for the project to work were minimal. This makes the project self contained and easier to deploy both as a system service or as a Docker container.

The `src/webserver.c` implementation features:

- **Request Parsing:** A dedicated worker thread (`webserver_worker`) parses the raw HTTP request string using `sscanf` to extract the method and path.
- **Security:** The `serve_file` function enforces a root directory jail by checking for “.” substring presence, returning `403 Forbidden` if a traversal attempt is detected.
- **Concurrency:** A producer-consumer queue (`webserver_enqueue`) allows the main thread to offload file serving tasks to the webserver thread, preventing static file I/O from blocking the Mandelbrot rendering loop.

The Mandelbrot Set: Definition and Algorithms

The Mandelbrot set is defined as the set of complex numbers c for which the sequence $z_{n+1} = z_n^2 + c$ (starting with $z_0 = 0$) does not diverge to infinity. Mathematically, if $|z_n| > 2$, the sequence is guaranteed to diverge. The “escape time” algorithm iterates this formula up to a maximum limit; the number of iterations reached determines the pixel's color.

The algorithm is implemented in `src/mandelbrot.c`:

- **Scalar Loop:** The `render_scalar` function implements the classic escape time algorithm. It uses the optimization $x^2 + y^2 \leq 4.0$ to avoid expensive square root calculations.
- **Algebraic Simplification:** The update rule is expanded to $x_{\text{new}} = x^2 - y^2 + x_0$ and $y_{\text{new}} = 2xy + y_0$. By maintaining x^2 and y^2 as separate variables in the loop, the implementation minimizes the number of multiplication operations per iteration.

Threadpool Architecture

To utilize multi-core processors efficiently without the overhead of constantly creating and destroying threads, a threadpool was implemented. I decided not to use something like OpenMP APIs because, I wanted more flexibility than using OMP directives.

Implemented in `src/tpool.c`:

- **Synchronization:** The `tpool_t` structure uses a `pthread_mutex_t` to protect the task queue and a `pthread_cond_t` to signal sleeping worker threads when new work is available.
- **Task Queue:** A circular buffer holds function pointers (`thread_task_t`). When the main loop needs to render an image, it decomposes the frame into horizontal strips and pushes these tasks to the pool via `tpool_add_work`.
- **Worker Loop:** Threads run an infinite loop (`thread_worker`), blocking on the condition variable until a task is available, executing it, and then returning to sleep, ensuring low latency and high CPU utilization.

SIMD and Parallel Computation

To speed up the mandelbrot computation I decided to employ the techniques learned attending the *Distributed systems: paradigms and models* course. Single Instruction, Multiple Data (SIMD) allows a processor to perform the same operation on multiple data points simultaneously on the same ALU. The project leverages AVX2 (Advanced Vector Extensions) to process four double-precision floating-

point numbers at once. To maximize compatibility, the instruction sets supported are AVX, AVX2 and AVX-512, but unfortunately I do not have the hardware to test the latter instruction set, so I only audited the code manually.

The `render_simd` function in `src/mandelbrot.c` holds the SIMD implementation of the mandelbrot rendering:

- **Vectorized Math:** It uses intrinsic functions like `_mm256_mul_pd` and `_mm256_add_pd` to perform the complex number arithmetic on four pixels simultaneously (`_m256d`).
- **Conditional Masking:** Since different pixels diverge at different speeds, standard `if/break` logic cannot be used. Instead, `_mm256_cmp_pd` creates a comparison mask. The iteration count is updated using `_mm256_sub_epi64` only for those elements in the vector that satisfy the condition ($|z|^2 \leq 4$).
- **Packing:** The final 64-bit integer counters are packed down to 16-bit integers using permutation (`_mm256_permutevar8x32_epi32`) and packing (`_mm_packus_epi32`) instructions for efficient memory storage.

OpenGL and GPGPU Abstraction

Finally, I wanted to dive in GPU programming as I never tried to run any algorithm on a graphic accelerator. To implement this part I instructed the agent to use OpenGL since I knew it supports multiple GPUs vendors, meaning that with one algorithm I could render the mandelbrot both on NVIDIA and Intel GPUs (the one I have in my laptop). OpenGL is a cross-language, cross-platform API for rendering vector graphics. While primarily designed for visual output, it can be used for General-Purpose Computing on Graphics Processing Units (GPGPU).

The system abstracts the GPU hardware in `src/mandelbrot.c`:

- **Compute Shaders:** A GLSL compute shader (`#version 430 core`) handles the heavy lifting. It runs one thread per pixel (`gl_GlobalInvocationID`) and implements the Mandelbrot logic, including a geometric optimization that skips calculations for points inside the main cardioid.
- **Persistent Mapping:** To minimize driver overhead, the project uses `glMapBufferRange` with the `GL_MAP_PERSISTENT_BIT`. This maps GPU memory directly into the CPU's address space (`gl_mapped_buffer`), allowing the application to read back render results without explicit buffer copy commands.
- **Vendor Agnostic:** The `init_opengl_for_vendor` function dynamically scans available EGL devices to find and initialize the correct GPU (Intel or NVIDIA), ensuring the application runs on the desired hardware backend.

JavaScript 2D Canvas and Rendering

Finally, regarding the frontend I decided not to use a framework such as React that I already know and use, but to develop the webapp using only HTML, CSS and JavaScript so that I could learn the basics of web development, which are hidden by the abstractions provided by the modern frameworks. Also, I did not opt for a native GUI because it cannot be easily distributed across different operating systems and architectures, while a browser is available on almost all platforms. Also, this way the rendering and the visualization can happen on different machines. The webapp is very simple and features a sidebar with all the controls and a `<canvas>` element which provides a drawing surface where to paint the mandelbrot. For high-performance rendering, this project uses the `CanvasRenderingContext2D` API. EXPLAIN BETTER WHY

The frontend logic in `script.js` handles the visualization:

- **Binary Protocol:** The client sends coordinates as a string and receives a raw binary `ArrayBuffer` containing 16-bit iteration counts.

- **Client-Side Coloring:** To allow instant theme switching without re-requesting data, the mapping from iteration count to RGBA color happens in the browser. A Uint32Array palette is precomputed.
- **Buffer Management:** The received data is wrapped in a Uint16Array. The script iterates through this array, looks up the color in the palette, and writes the resulting 32-bit pixel value into an ImageData buffer's Uint32Array view.
- **Blitting:** The final image is pushed to the GPU-accelerated canvas using `ctx.putImageData` (or `createImageBitmap` for scaling), providing a smooth 60 FPS experience.